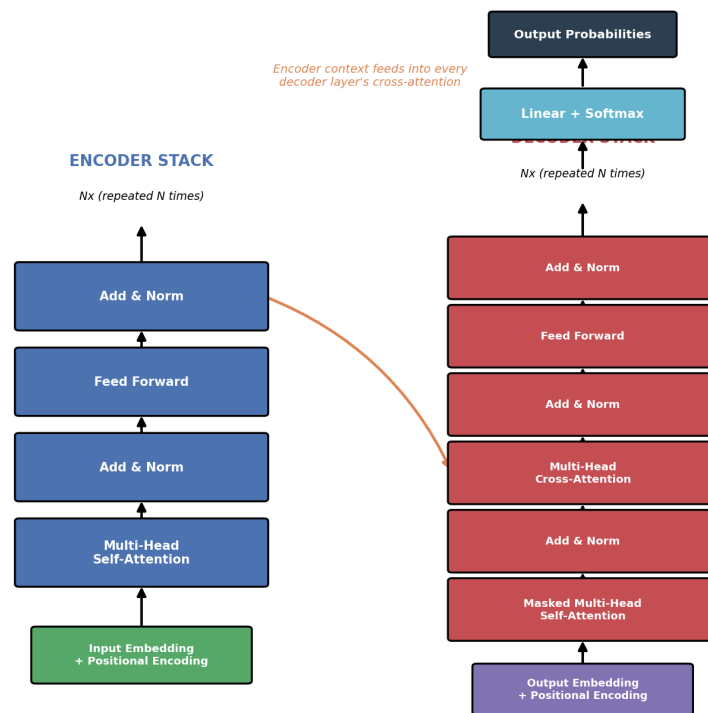


# Transformer Architecture

## Complete Guide

*Encoder, Decoder, Masked Attention & Autoregressive Generation Explained Visually*

### The Full Transformer Architecture



Covers: Understanding the architecture • Encoder architecture & feed-forward network • Masked self-attention & masked multi-head attention • Transformer decoder • Autoregressive models • Why generation can't be parallelized

# 1. Understanding the Architecture

The Transformer, introduced in "*Attention Is All You Need*" (2017), has two main halves: an **encoder** stack that reads and understands the input sequence, and a **decoder** stack that generates the output sequence, one token at a time, using both its own previous outputs and the encoder's understanding of the input.

## The Full Transformer Architecture

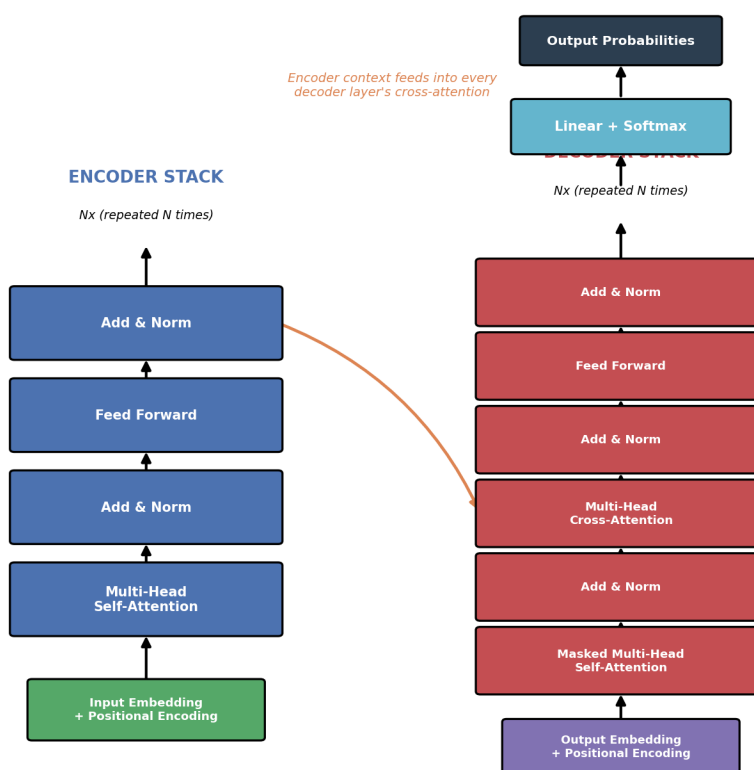


Figure 1: The complete Transformer - an encoder stack on the left feeding into a decoder stack on the right.

- **Encoder:** takes the input sequence (e.g. a sentence in French) and produces a rich, context-aware representation of it.
- **Decoder:** generates the output sequence (e.g. the English translation) step by step, attending both to its own previously generated tokens AND to the encoder's output.
- Both stacks are made of  $N$  identical layers stacked on top of each other (commonly  $N=6$  in the original paper).

## 2. Encoder Architecture

---

Each encoder layer has exactly two sub-layers, each wrapped with a residual connection followed by Layer Normalization: a multi-head self-attention sub-layer, and a feed-forward sub-layer.

### Encoder Layer - Detailed View

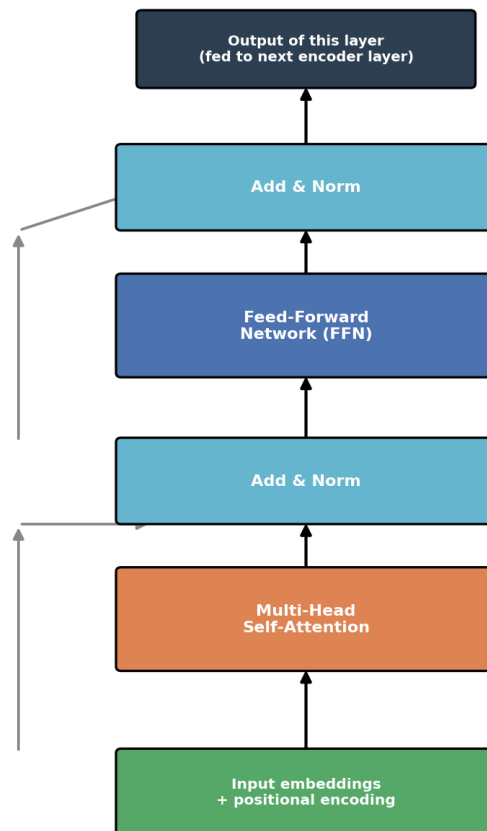


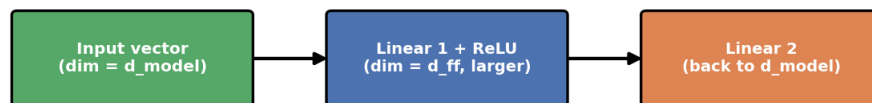
Figure 2: One encoder layer in detail - self-attention and feed-forward, each followed by Add & Norm.

- **Multi-Head Self-Attention:** lets every word in the input attend to every other word, building context-aware representations (see the separate Self-Attention & Multi-Head Attention guide for full details).
- **Add & Norm:** a residual (skip) connection adds the sub-layer's input back to its output, then Layer Normalization stabilizes the result. This residual path makes it much easier to train deep stacks of layers.
- **Feed-Forward Network:** a small, position-wise fully-connected network applied identically to every position.

## 2.1 Encoder - The Feed-Forward Network

After attention has mixed information across positions, the feed-forward network (FFN) processes each position's vector independently, adding extra non-linear transformation capacity. It consists of two linear layers with a ReLU activation in between, and its hidden dimension is typically much larger than the model's main dimension (e.g. 2048 vs 512).

### The Feed-Forward Network (Position-wise)



$$\text{FFN}(\mathbf{x}) = \text{ReLU}(\mathbf{x} \cdot \mathbf{W1} + \mathbf{b1}) \cdot \mathbf{W2} + \mathbf{b2}$$

*Applied IDENTICALLY and INDEPENDENTLY to every position/token -  
the same two weight matrices are reused for every word in the sequence*

Figure 3: The feed-forward network - expand, apply non-linearity, then project back down - applied identically to every token position.

$$\text{FFN}(\mathbf{x}) = \text{ReLU}(\mathbf{x} \cdot \mathbf{W1} + \mathbf{b1}) \cdot \mathbf{W2} + \mathbf{b2}$$

Note that this is where most of the Transformer's actual parameters live - self-attention mixes information ACROSS positions, while the FFN adds representational depth WITHIN each position.

### 3. Masked Self-Attention

---

The decoder can't use regular self-attention the way the encoder does. When generating text, the model must predict each word using only the words that came before it - it would be cheating (and would break at inference time) if it could "peek" at future words it hasn't generated yet. **Masked self-attention** enforces this by blocking each position from attending to any future position.

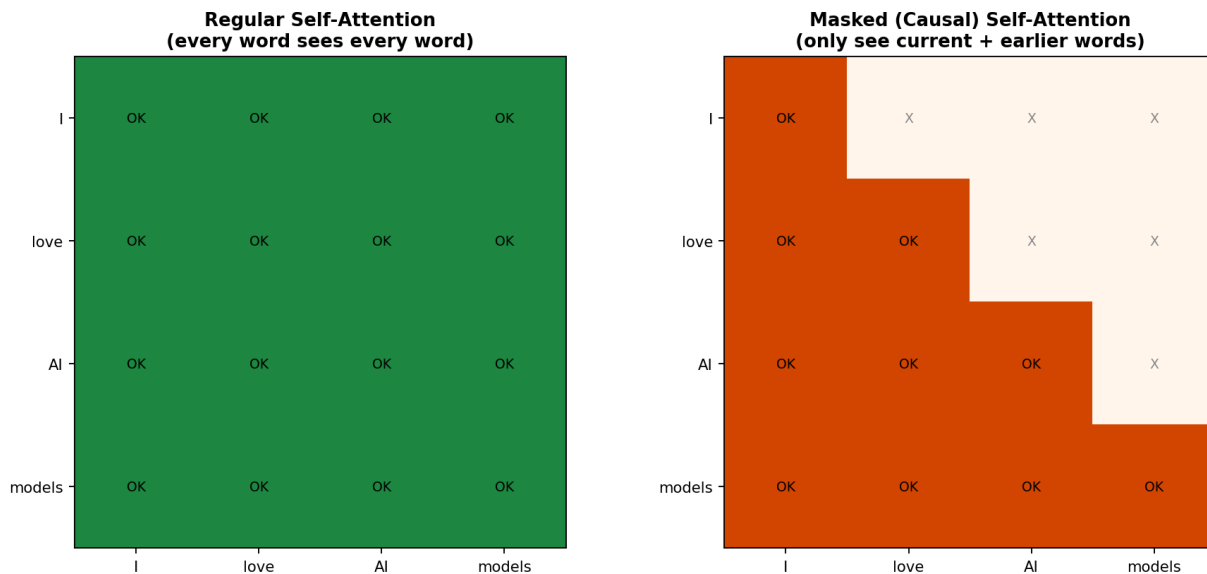


Figure 4: Regular self-attention lets every word see every word (left); masked self-attention only lets each word see itself and earlier words (right) - this triangular pattern is called a 'causal mask'.

### 4. Masked Multi-Head Attention in the Transformer

---

Mechanically, the mask is applied INSIDE the scaled dot-product attention computation, right before the softmax step: positions that shouldn't be attended to have their score set to negative infinity, so that after softmax, their attention weight becomes exactly zero.

### Masked Multi-Head Attention - How the Mask Is Applied

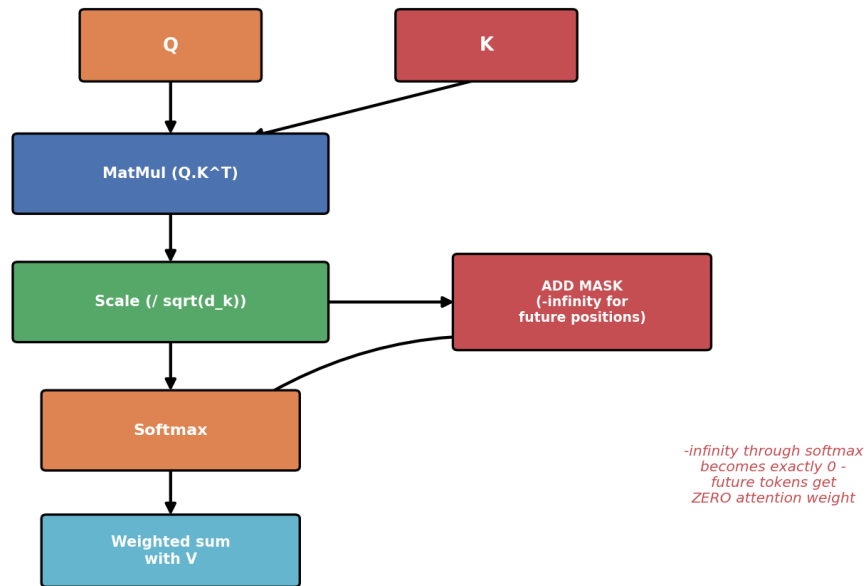


Figure 5: The mask is added right before softmax - negative infinity scores collapse to zero attention weight after softmax.

**MaskedAttention(Q,K,V) = softmax(  $QK^T/\text{sqrt}(d_k) + \text{Mask}$  ) · V, where Mask has -infinity in future positions**

This is applied across all heads in the decoder's first attention sub-layer - hence "masked MULTI-HEAD attention." Every head independently respects the same causal mask.

## 5. Transformer Decoder

Each decoder layer has THREE sub-layers (one more than the encoder), again each wrapped with residual connections and Layer Normalization:

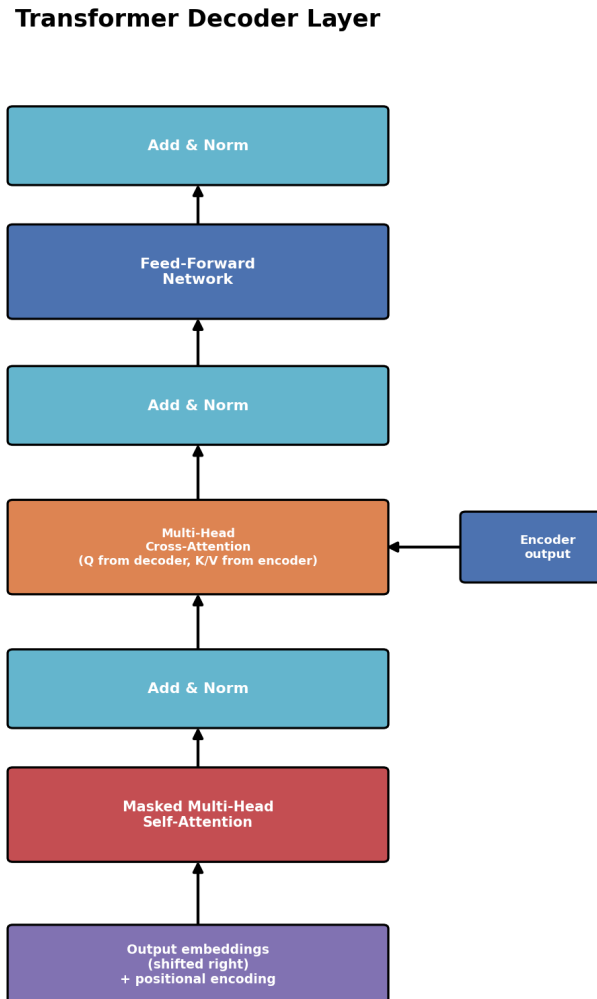


Figure 6: One decoder layer - masked self-attention, cross-attention to the encoder, then feed-forward.

- **Masked Multi-Head Self-Attention:** lets each output position attend to earlier output positions only (as described above).
- **Multi-Head Cross-Attention:** the Query comes from the decoder, but the Key and Value come from the ENCODER's output - this is how the decoder incorporates information about the input sequence while generating the output.
- **Feed-Forward Network:** identical in structure to the encoder's FFN.

The final decoder output is passed through one Linear layer and a softmax, producing a probability distribution over the entire vocabulary for the next token.

## 6. Autoregressive Models

An **autoregressive** model generates output one step at a time, where each new prediction depends on all the previous outputs. In language modeling, this means predicting the next word given every word generated so far, then feeding that new word back in as part of the input for predicting the following word.

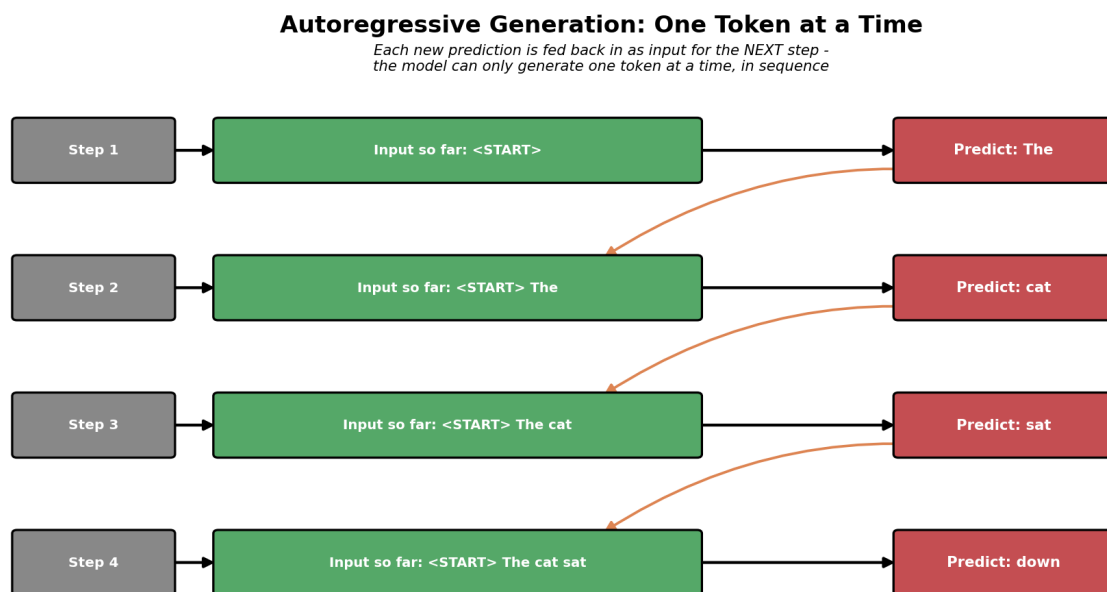


Figure 7: Autoregressive generation - each newly predicted token gets appended to the input before predicting the next one.

## 7. The Transformer as an Autoregressive Model

The Transformer's decoder (and decoder-only models like GPT) is trained to be autoregressive: masked self-attention ensures that, during training, the model learns to predict each token using only earlier tokens - exactly matching how it will need to behave during actual text generation, where future tokens genuinely don't exist yet.

This consistency between training behavior and generation behavior is precisely why masking matters so much: it makes training and inference behave the same way with respect to information flow, even though training sees the whole target sequence at once.



## 8. The Problem with Parallelizing Generation

One of the Transformer's biggest selling points is that **training** is highly parallelizable - since the entire target sequence is already known during training, masked self-attention can process all positions simultaneously in one forward pass (the mask just prevents information leakage from the future). But **generation (inference)** is a completely different story.

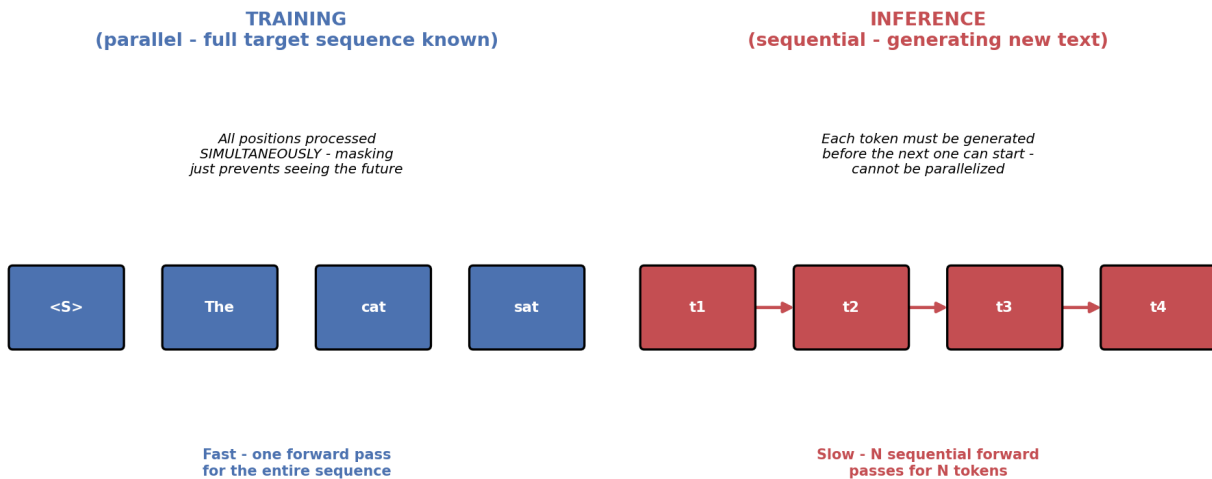


Figure 8: Training processes the whole sequence in parallel; inference must generate one token at a time, sequentially.

At generation time, the model doesn't know the next word yet - it has to actually predict it first, before it can even start working on the word after that. This means autoregressive generation is fundamentally **sequential**: to generate N tokens, you need N separate forward passes through the decoder, each one waiting for the previous token to be produced.

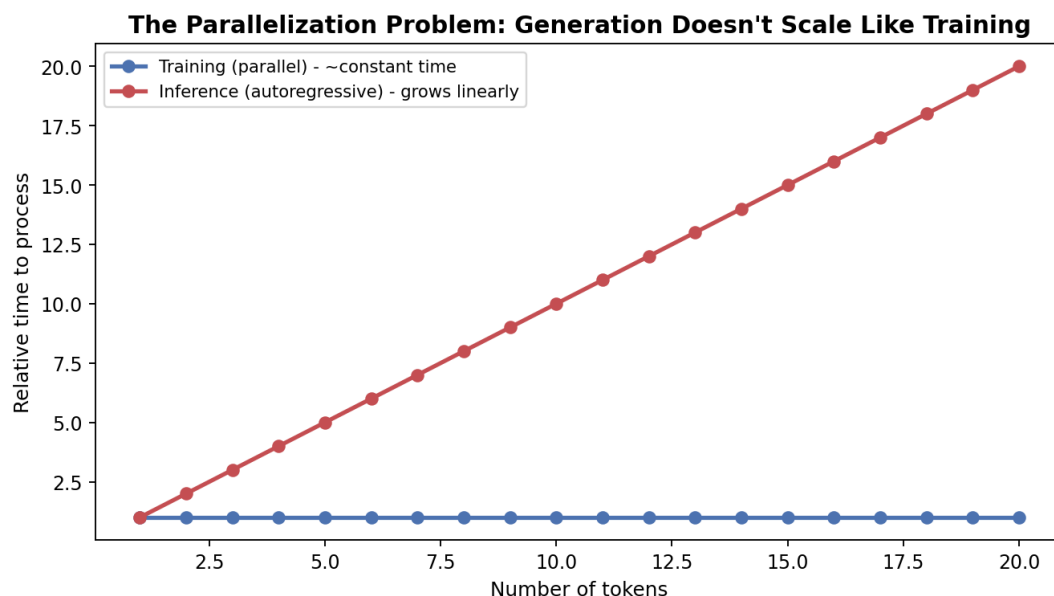


Figure 9: Training time stays roughly constant regardless of length (parallel), while inference time grows linearly with the number of tokens generated (sequential).

This is the core trade-off of autoregressive Transformers: they train efficiently in parallel, but generate text slowly, one token at a time. This limitation has motivated a lot of ongoing research into faster decoding strategies - such as speculative decoding, parallel sampling techniques, and non-autoregressive generation methods - to reduce the sequential bottleneck at inference time.

## Summary

| Concept                     | Key Idea                                                                       |
|-----------------------------|--------------------------------------------------------------------------------|
| Encoder                     | Self-attention + FFN, builds context-aware representation of the input         |
| Feed-forward network        | Position-wise 2-layer network, applied identically to every token              |
| Masked self-attention       | Blocks each position from attending to future positions (causal mask)          |
| Masked multi-head attention | The causal mask applied across all attention heads in the decoder              |
| Decoder                     | Masked self-attention + cross-attention (to encoder) + FFN                     |
| Autoregressive models       | Generate one token at a time, each depending on all previous tokens            |
| Parallelization problem     | Training is parallel (target known); generation is sequential (target unknown) |

The Transformer's genius lies in this balance: parallel, attention-based training for speed, and a masked, autoregressive decoding process that guarantees the model behaves consistently whether it's learning from a full sequence or generating one word at a time in the real world.